

Time Flies, Until It Doesn't: Predicting Flight Departure Delays

Phase 3: December 13, 2025

Abstract

Flight delays impose substantial costs on travelers in terms of time, money, and coordination. To address this, we developed a predictive engine for a traveler-facing mobile application that provides flight-specific delay estimates up to two hours before departure. Delays are defined as departures occurring 15 minutes or more after the scheduled time, and the problem is framed as a regression task to produce granular estimates.

Our dataset combined U.S. Department of Transportation flight records with NOAA weather data from 2015–2019. The pipeline included a spatial-temporal join of airports to weather stations, robust data cleaning to prevent leakage, and engineered features spanning temporal, weather-derived, flight-history, and graph-based network characteristics. Model performance was evaluated using a rolling-window cross-validation strategy, training on 2015–2018 data and holding out 2019 as the final test set.

We compared three model families: a linear regression baseline, a Multi-Layer Perceptron (MLP) with embeddings for high-cardinality features, and Gradient-Boosted Trees (XGBoost). While linear regression and MLP provided reasonable estimates, they struggled with the skewed, zero-inflated delay distribution. To overcome this, we developed a two-stage XGBoost model with a classifier for delay probability and a regressor for delay magnitude, further refined with a joint evaluation strategy to penalize errors.

The final two-stage XGBoost model achieved the best performance, with a 2019 test set MAE of 9.51 minutes and RMSE of 40.76 minutes, significantly outperforming the linear baseline (MAE: 14.43 min) and single-stage XGBoost (MAE: 11.31 min). This demonstrates the model's ability to accurately predict typical delays while effectively capturing rare, severe delays in the long tail.

Introduction

Flight delays cost travelers time, money, and coordination challenges. To address this, we develop a predictive engine for a traveler-facing app that provides flight-specific delay estimates two hours prior to scheduled departure. Specifically, the app will notify users of a potential delay for their upcoming flight. We define a delay as a departure occurring 15 minutes or more after the scheduled time. We define prediction time, when the model has to make its prediction, as two hours before scheduled departure time. Framing the task as a regression problem enables the model to produce granular delay estimates rather than a binary “delayed/not delayed” label. Training data combines U.S. Department of Transportation flight records from 2015–2019 with weather data from the National Oceanic and Atmospheric Administration (NOAA).

We evaluate model performance primarily using Mean Absolute Error (MAE), which provides a direct, interpretable measurement of the average error in minutes. MAE reflects the typical user experience without being disproportionately skewed by extreme outliers. However, because severe delays represent

the highest operational impact, we utilize Root Mean Squared Error (RMSE) as a secondary metric. RMSE serves as an indicator of our model’s performance on the “long tail” of severe delay events, ensuring that they are not ignored by the optimization process.

End-to-End Pipeline

This project follows a fully end-to-end machine learning pipeline designed to produce flight-specific departure delay predictions using only information available two hours prior to scheduled departure. Figure 1 summarizes the complete workflow from raw data ingestion through final model evaluation.

The pipeline consists of the following stages:

1. Raw data ingestion: US DOT flight records, NOAA weather observations, and airport metadata
2. Flight-weather join: spatial matching of airports to weather stations and temporal alignment of hourly observations.
3. Data cleaning and leakage control: removal of high missingness features and cancelled flights
4. Feature engineering: construction of time-based, weather derived, flight-history, and graph-based features
5. Model families explored: linear regression, gradient-boosted trees (XGBoost), and neural networks (MLP)
6. Evaluation strategy: rolling-window cross-validation with MAE as the primary metric and RMSE as the secondary metric.

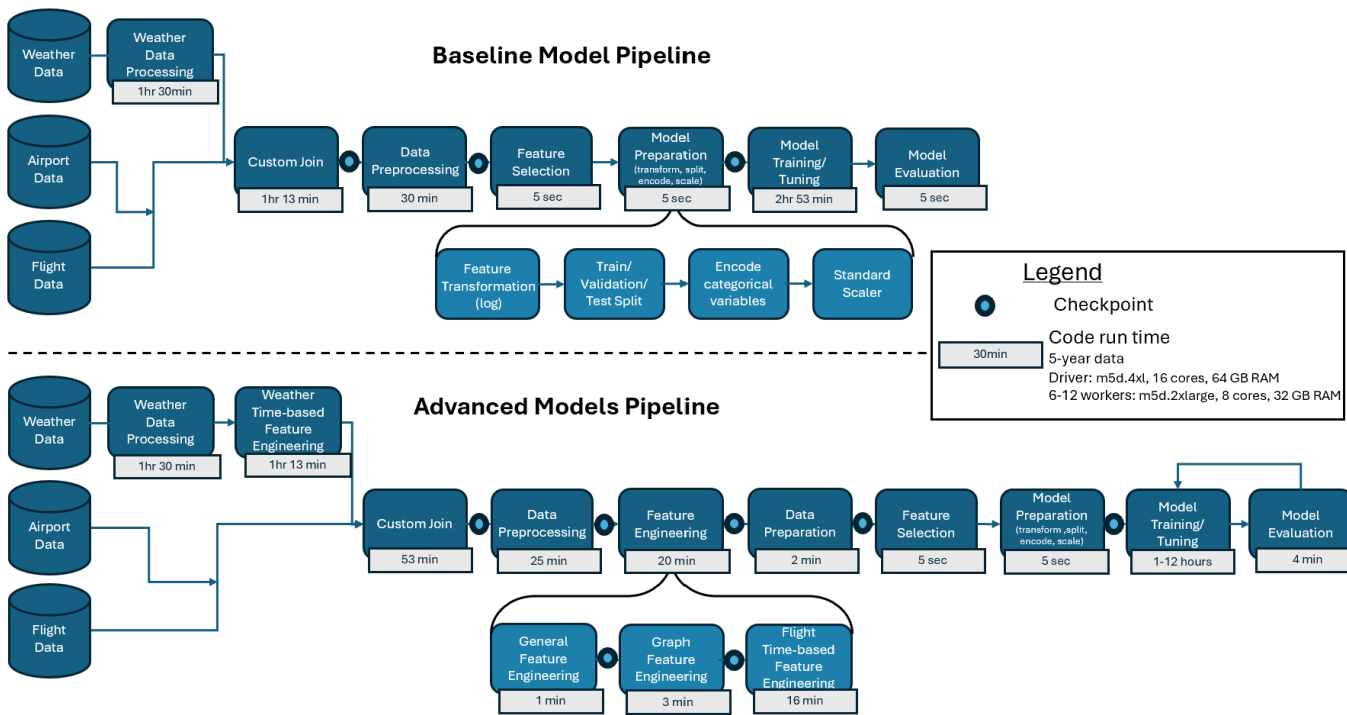


Figure 1. The full model pipeline with approximate run times on the 5-year data.

Data Lineage & Processing

Data Description

We used raw data from three different types of tables: flights, weather, and airports. The flights dataset contained detailed records of every scheduled US commercial flight from 2015 to 2019, with fields describing scheduling, operations, and performance. The weather dataset contained hourly measurements of weather conditions such as precipitation, temperature, etc., as well as daily and monthly summary statistics, from weather stations across the U.S. between 2015 and 2019. Finally, the airports datasets contained information about their location and nearby weather stations, useful for our custom join, detailed in the Flight-Weather Data Join section.

Data Size and Source

Table 2. Shapes and sizes of 2015-2019 data

Data	Source	Rows	Columns	Total Size
Flights (raw, deduped)	US Department of Transportation	31.74M	109	2.41 GB
Weather (raw)	NOAA Local Climatological Data	630.9M	177	23.35 GB
Airports (raw)	Open Airports Database	6,138	11	260 MB
Custom Join	-	31.74M	249	8.04 GB

Missing Data Handling and Duplicate Processing

We did the following to handle missing data and process duplicates:

- Dropped all rows with no departure delay
- Dropped all columns with more than 80% null
- Deduped flight records in flight data
- Imputed pressure change from consecutive pressure readings

Flight-Weather Data Join

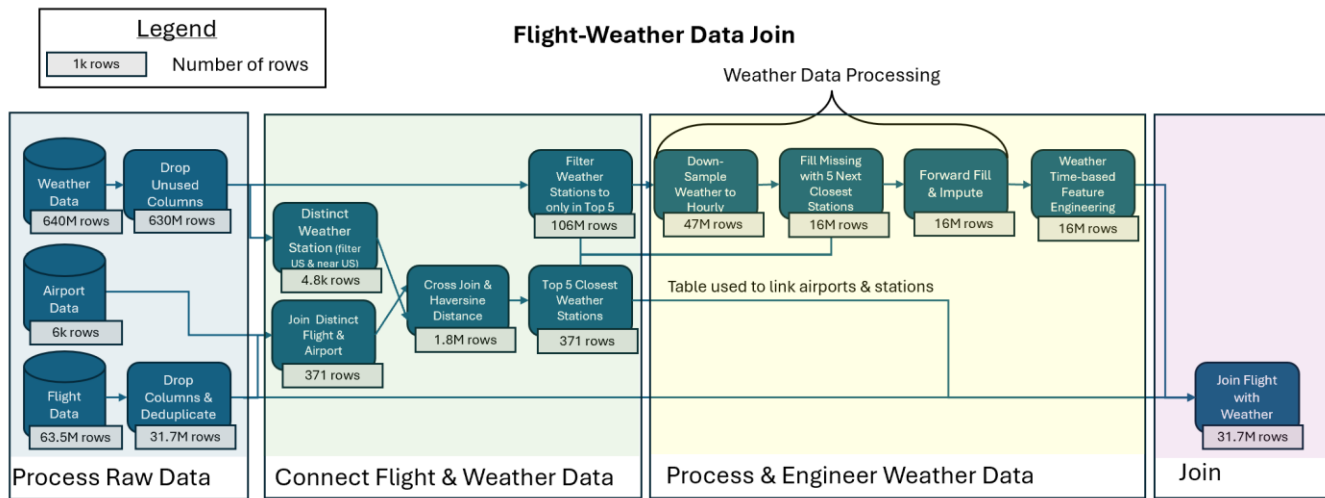


Figure 2. The pipeline for our join between the weather, airport, and flight data.

We joined each origin airport to its nearest weather station by augmenting flight records with latitude, longitude, and time zone information from an external airport dataset and identifying the closest weather station using haversine distance. We restricted cross-join computations to the distinct sets of airports and weather stations to ensure scalability.

Because hourly weather observations were irregular and sometimes duplicated within the same hour, we standardized all stations to a single reading per hour by selecting the last non-null observation, which is closest to the prediction time. To prevent data leakage, each flight was joined to weather data from three hours prior to its scheduled departure, ensuring all weather inputs occurred strictly before the prediction window.

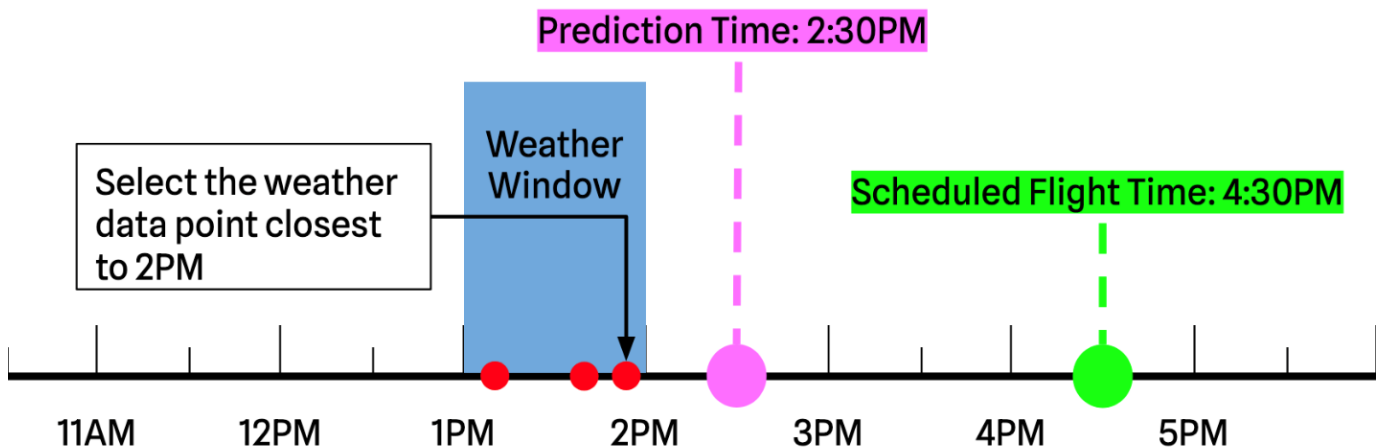


Figure 3. Method for selecting weather observations when joining weather data to flights, designed to prevent data leakage.

Finally, we imputed missing weather values with observations from the four nearest weather stations within 50km, forward-filled the rest within a six-hour limit and, if necessary, imputed with an average from the last 3 days.

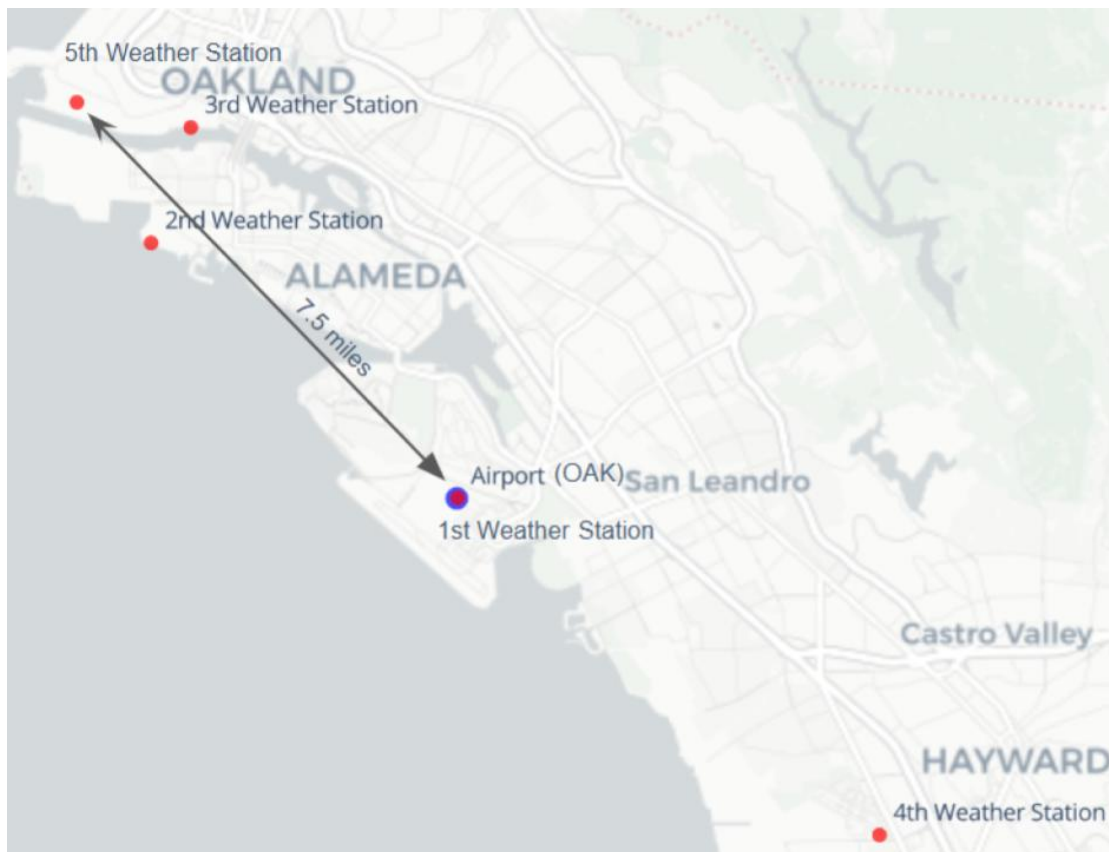


Figure 4. The 4 closest weather stations to Oakland Airport (OAK), used to fill in missing weather values for flights departing OAK

The five-year join pipeline required approximately 53 minutes.

Exploratory Data Analysis

Target Variable EDA

Departure delays are highly right-skewed, with most flights having no delays. Approximately 80% of flights experience no delay or a delay of less than 15 minutes. This imbalance motivates both transformation of the target variable during modeling and strategies to prevent the model from defaulting to predicting near-zero delay.

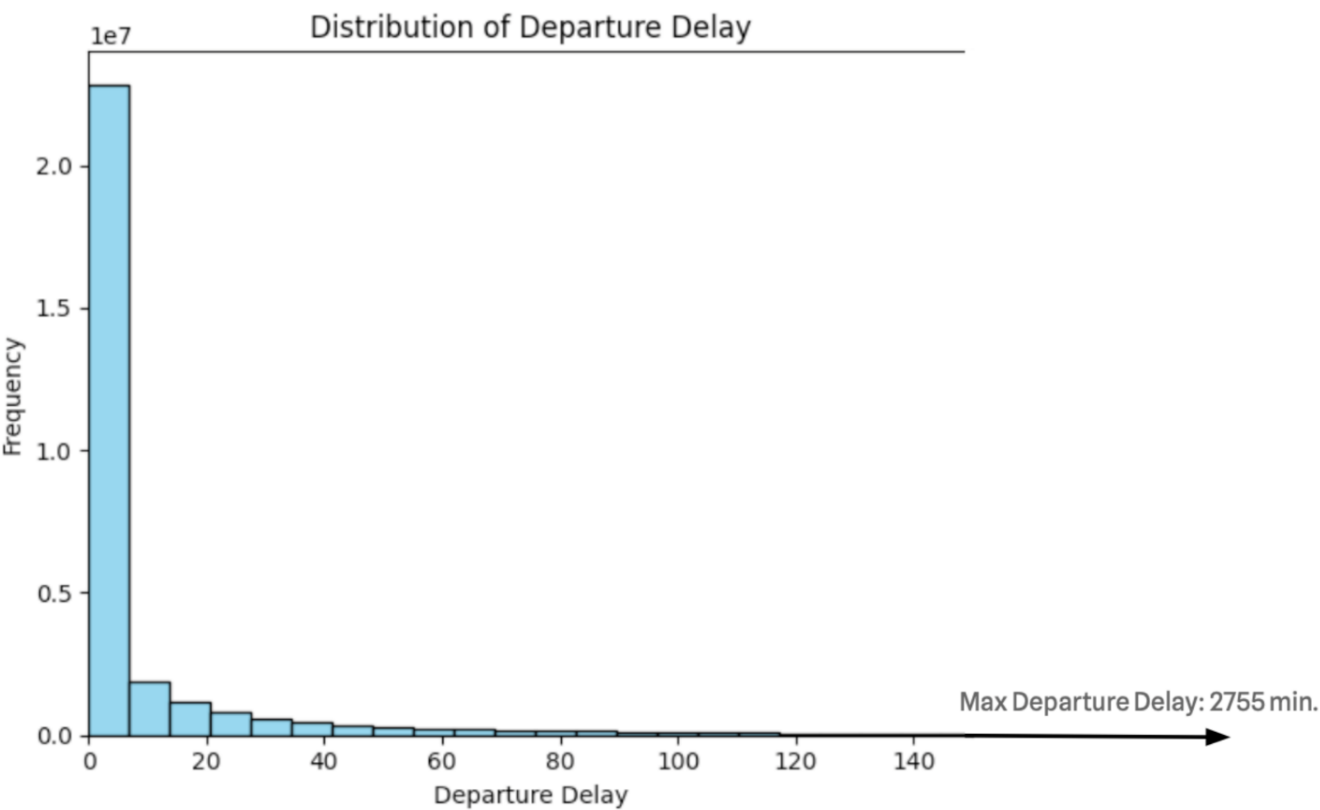


Figure 5. Distribution of departure delay with the x-axis truncated at 150 minutes. The maximum departure delay was 2755 minutes.

Feature Family EDA

Weather-related features spanned a wide range of distributions, scales, and physical units (e.g., inches of precipitation, miles per hour for wind speed, and miles of visibility). Some variables are highly skewed with heavy tails while others are more tightly concentrated. These differences mean that raw numeric values are not directly comparable across features and require appropriate scaling to place them on a common footing before modeling.

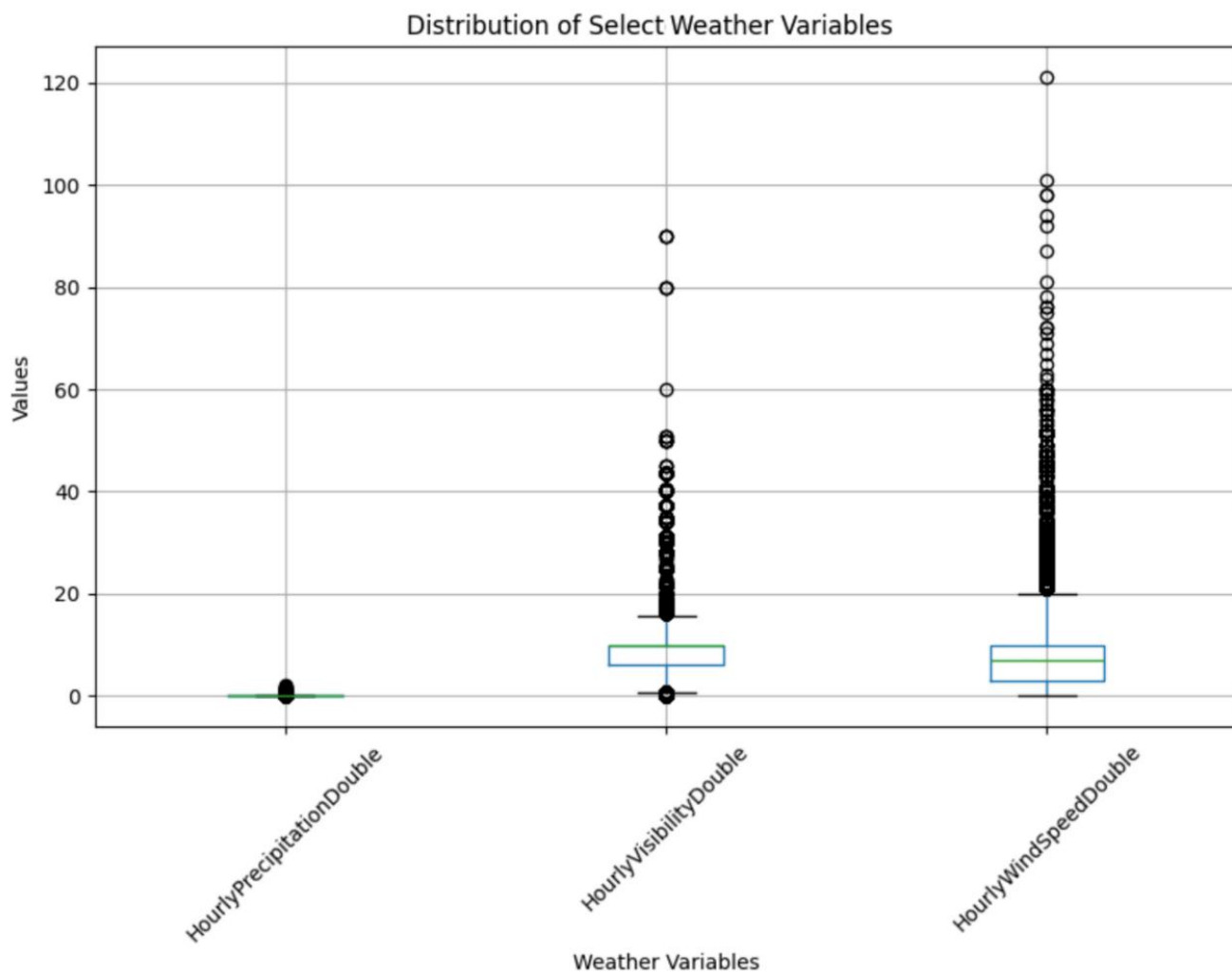


Figure 6. Boxplots of select weather variables; disparate scales are apparent.

To assess monotonic relationships while remaining robust to outliers, we compute pairwise Spearman rank correlations between features and the target variable. We observe weak correlations, reinforcing the need for models that can capture interactions across weather, temporal, and network features.

Spearman's Rank Correlation Between Departure Delays and Weather Features

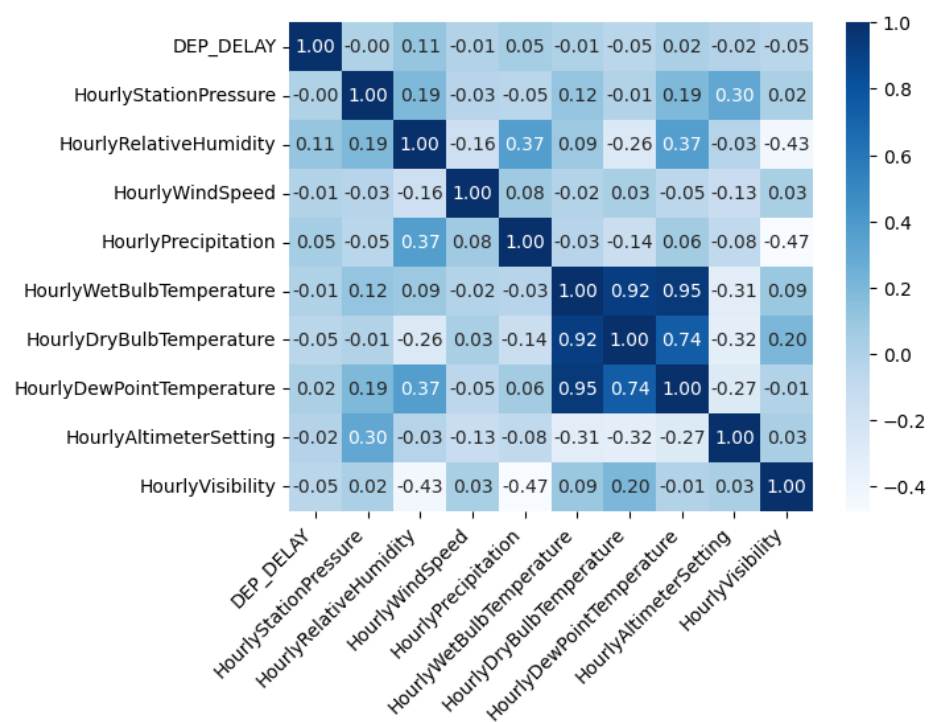


Figure 7. Spearman's rank correlation between departure delay and select weather features.

Motivated by the intuition that delays propagate across consecutive flights, we examined the relationship between a flight's departure delay and the delay of its previous leg. Figure 8 shows a strong association, especially when turnaround times are short. Three patterns emerge: near one-to-one delay carryover for tight turnarounds, minimal propagation when turnaround gaps are long, and late departures following on-time arrivals, indicating exogenous factors such as weather or airport congestion.



Figure 8. Relationship between previous departure delay and current departure delay.

Feature Engineering

Feature Families Overview

The weak correlations of the raw features during EDA necessitated a significant feature engineering effort. We organized engineered features into four families: 1. time based 2. weather derived 3. flight history and 4. graph based, each designed to capture a distinct source of delay variability (Table 3).

Table 3. 5-Year Feature Set

Feature Family	Count
Time/ Date	35
Airplane	5
Airport	29
Weather Metrics	70
Total	139

Time-Based & Flight-History Features

Time based features encode recurring temporal patterns such as hour-of-day, day-of-week, seasonality, and holidays. Flight history features summarize recent aircraft operations to capture short-term operational dependencies.

Previous flight information is merged using tail number and airline identifiers. To prevent data leakage, only flights that departed strictly before the prediction time are used when computing these features.

Table 4. Flight-Related Engineering Features

Feature	Feature Type	Feature Description
Previous Flight Information	Numerical	The scheduled turnaround time and the recent delay history of the aircraft assigned to the flight
Recent Average Delay	Numerical	24-hour, 7-day (prior to prediction time) average of delays grouped by Origin, Destination, Previous Origin, Tail Number, and Flight Number
Daily Scheduled Flights	Numerical	The total scheduled flights for the origin on this date
Holiday	Boolean	Whether the date is a holiday
Holiday Window	Boolean	Whether the date falls into a holiday window. Major holidays such as Thanksgiving and Christmas have a +/- 4-day window (4 days before, 4 days after), while minor holidays such as Columbus Day have a +/- 1-day window. The weekend days prior to a Monday holiday are also considered within a holiday window.

Weather-Derived Features

We derive rolling accumulations, averages, and rates of change across multiple time horizons to capture both sustained conditions and rapidly deteriorating weather. Wind direction is encoded using sine and cosine transformations, and precipitation type is represented with binary indicators.

Table 5. Weather-Related Engineering Features

Feature	Feature Type	Feature Description
Accumulation of precipitation	Numerical	Sum of precipitation over the last # hours # = 3, 6, 12, 24
Rate of change of precipitation	Numerical	The difference between the precipitation in the last # hour(s) and the prediction time # = 1, 2, 3
Maximum pressure drop	Numerical	Largest pressure drop over the last 24 hours
Average wind speed	Numerical	Average of wind speed over the last # hours # = 3, 6, 24
Rate of change of wind speed	Numerical	The difference between the wind speed in the last # hour(s) and the wind speed time # = 1, 2, 3
Average recent visibility	Numerical	Average of visibility over the last # hours # = 3, 6, 12
Rate of change of visibility	Numerical	The difference between the visibility in the last # hour(s) and the wind speed time # = 1, 2, 3
Current weather type	Categorical	Conditions at time of prediction: Rain, Snow, Fog, Hail, Dust/Sand, Smoke/Haze, Tornado
Hourly wind direction	Numerical	Added 2 columns (sine and cosine) using sinusoidal transformation

After creating the boolean indicator for each weather type, we compared the average delay for flights that did or did not experience that particular weather condition (Figure 9). Tornado activity and hail events show the most extreme difference in associated delay time. The presence of dust/ sand or smoke/ haze appears to have no effect on the average delay.

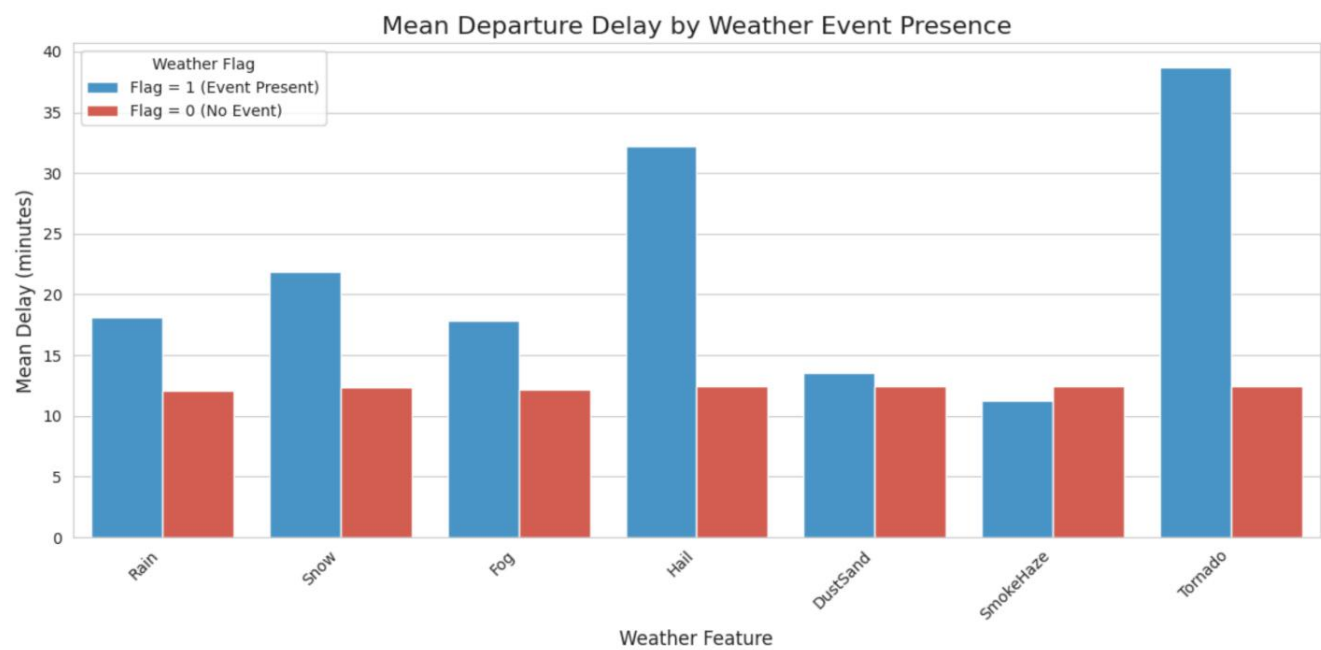


Figure 9. Average departure delay comparisons for different weather conditions

Graph-Based Features

We modeled the airport network as a weighted directed graph, with airports as nodes and flight volumes as edges, to capture network-level delay propagation (Table 6).

Table 6. Graph-based engineered features

Feature	Feature Type	Feature Description
Weighted degree centrality	Numerical	The sum of all flight counts (edge weights) connected to an airport. Serves as a proxy for airport congestion.
In/out centrality	Numeric	The sum of flight counts specifically arriving (In) or departing (Out) an airport. Distinguishes an airport’s role as a delay absorber (high In-degree) versus a delay generator (high Out-degree)
Louvain Modularity	Categorical	Cluster ID assigned by optimizing modularity (density of links inside vs. outside communities) on the flight graph. Captures regional structure by grouping airports that frequently exchange traffic.

Airport Network — Node Size by Degree Centrality

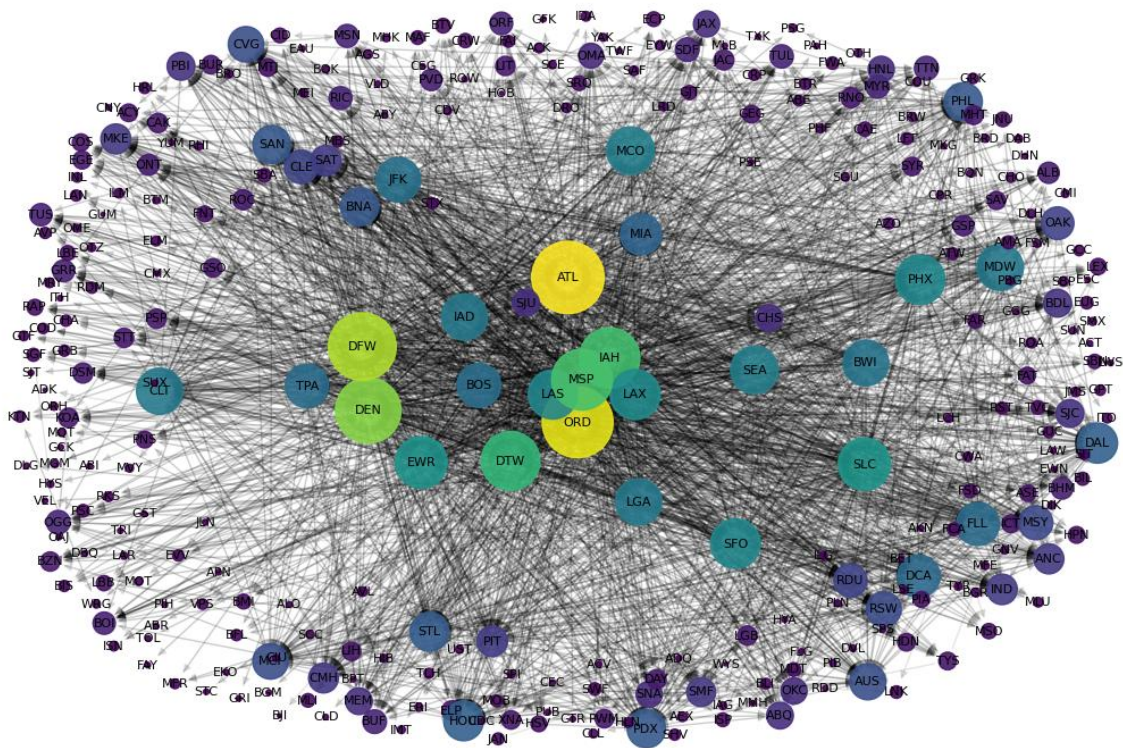


Figure 10. Degree centrality visualization (2015). Node size represents the weighted degree of each airport.

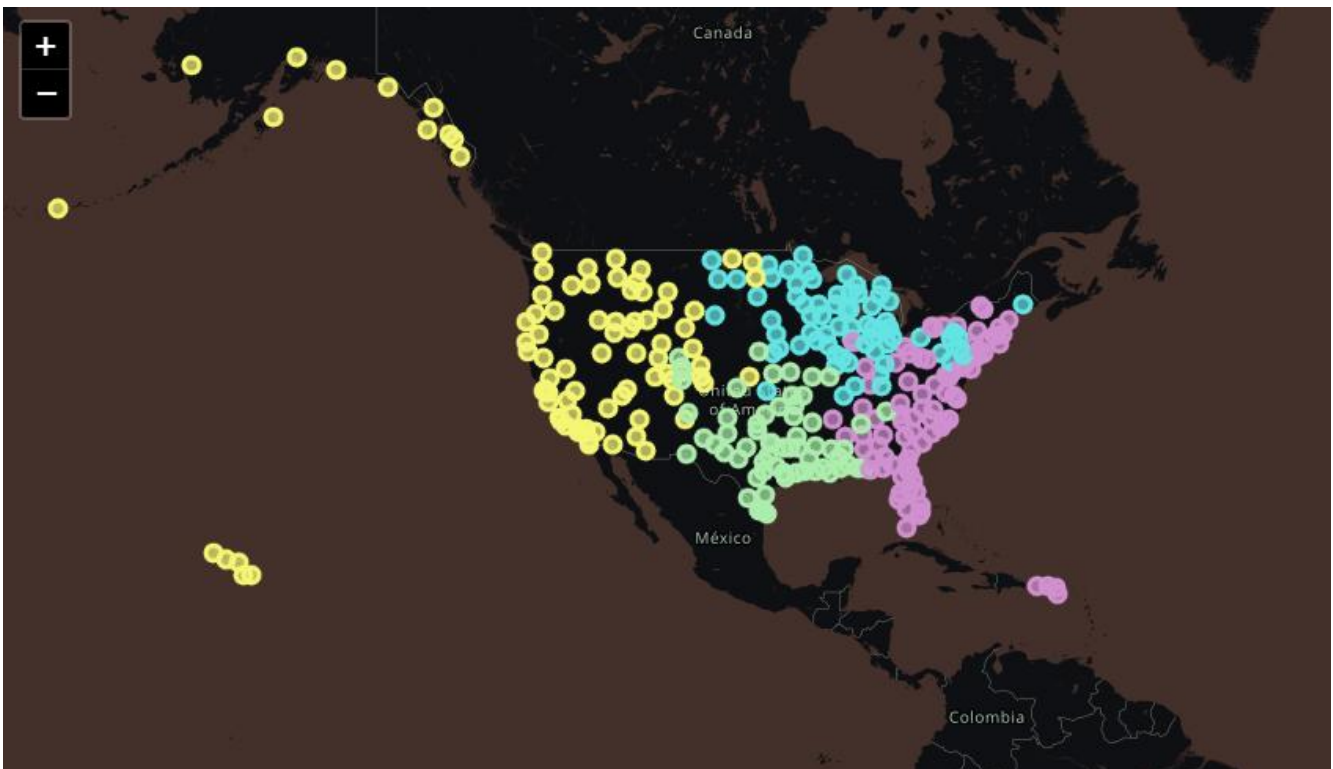


Figure 11. Louvain modularity (2015). Different colors represent airports within a particular community.

Encoding & Scaling

Cyclical temporal features (e.g., hour of day, day of week, quarter) and wind direction are encoded using sinusoidal transformations, with sine and cosine components added for each variable.

Table 7. List of cyclically encoded features

Feature	Feature Type	Encoding Type
Time (Date, Hour, Day of Week, Day of Month, Quarter)	Numerical	Sinusoidal transformations
Hourly wind direction	Numerical	Sinusoidal transformations

Categorical variables are encoded using a hybrid strategy. Features with low cardinality (fewer than 10 unique values) are one-hot encoded, while higher-cardinality categorical features are binary encoded to limit dimensionality. For the MLP, select high-cardinality categorical variables (origin, destination, carrier, and tail number, flight number and state) are instead represented using learned embeddings, allowing the model to capture similarity structure between categories.

Table 8. Select features and their cardinality

Categorical Feature	Cardinality (Number of levels)	Encoding Type
Unique airports	342	Binary
Unique airlines	19	Binary
Origin and destination state/territories	54	Binary
Tail numbers	7,837	Learned Embeddings
Flight numbers	7,085	Learned Embeddings
Airport type	4	One Hot
Weather type	7	One Hot

To address the heavy right skew of departure delays observed during EDA, the target variable is log-transformed during training.

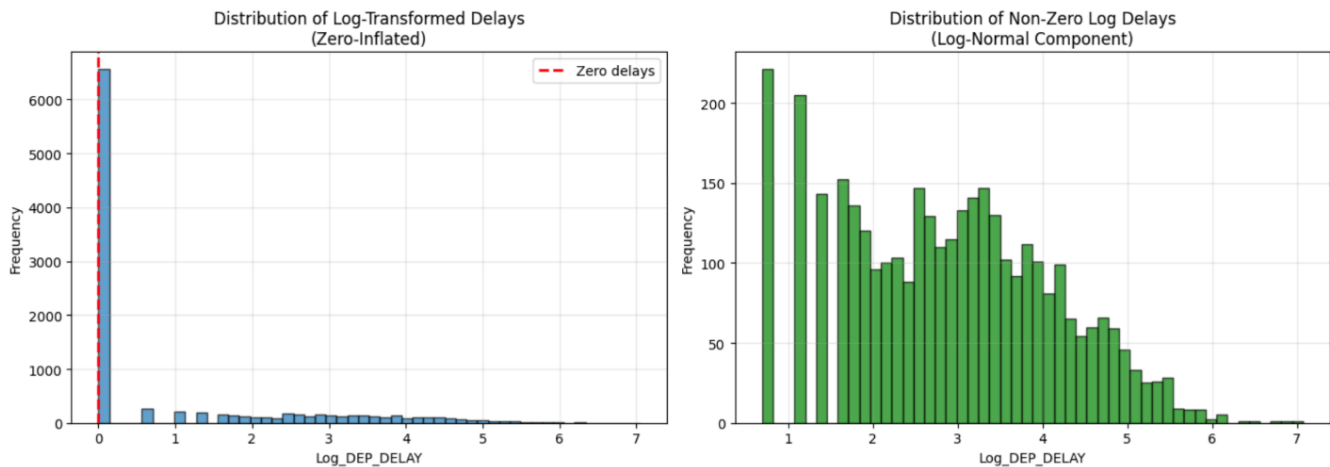


Figure 12. The graph on the left shows the distribution of departure delays post-log transform. The graph on the right is the same but removes delays equal to 0.

All continuous input features are standardized to zero mean and unit variance using a standard scaler. Identical encoding and scaling transformations are applied across training, validation, and test splits.

Updated Feature Correlations

We computed a Spearman correlation matrix to assess the monotonic relationship between engineered numerical features and departure delay. As shown in Figure 13, several engineered features exhibit stronger correlations with the target than the base features alone (refer to Figure 7), with prior-flight delay and recent aircraft- and route-level delay aggregates showing the highest associations (up to 0.27). These results support the intuition that delay propagation and short-term operational history provide meaningful predictive signal beyond static scheduling features.

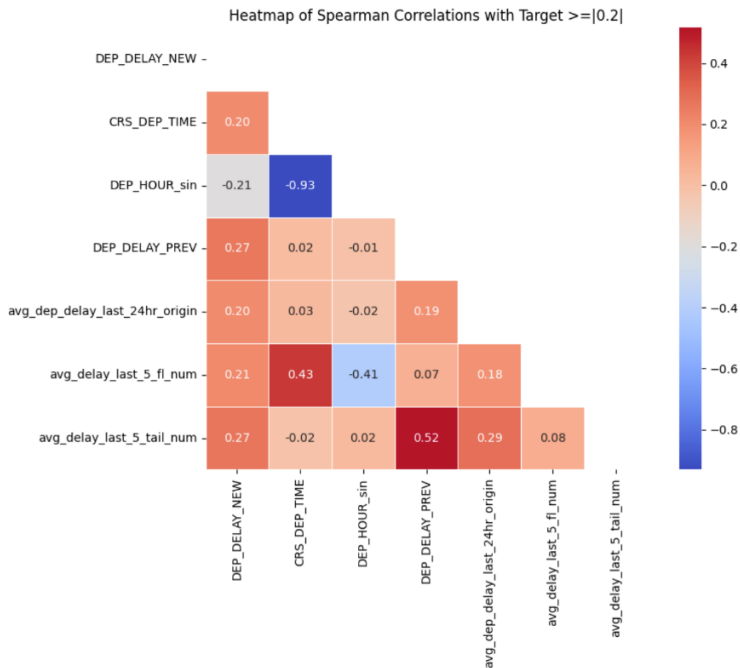


Figure 13. Spearman correlation matrix for top-ranking engineered features

Leakage Analysis

Data leakage occurs when information unavailable at prediction time is used during feature engineering, model training, or validation, resulting in overly optimistic performance estimates and poor generalization. In this project, an example of leakage would be computing average delays using flights that occurred after the departure time of the flight being predicted.

We mitigated data leakage by strictly separating train, validation, and test sets chronologically. All scaling parameters were fitted exclusively on training data, and the 2019 test set remained unseen during feature engineering and tuning. Similarly, weather and lag-based features were restricted only to information available prior to the prediction timestamp.

The only exception is the computation of graph-based features, which were calculated on the full dataset due to computational constraints. Due to the high stability of airport network structures and community memberships over time, we assess the impact of this leakage as minimal.

As a result, we expected reported performance metrics to reflect out-of-sample behavior.

Modeling Strategy & Evaluation Framework

Data Splits and Cross-Validation

For the full five-year dataset, we employed a rolling window 4-fold cross validation strategy. Each fold uses two years of training data and six months of validation data, with each fold offset by six months relative to the previous one (Figure 14). This produces folds covering 2015-2018, while 2019 is held out as the final test set.

Hyperparameters were tuned by averaging the validation MAE across all folds. Once optimal values were selected, the model was retrained on the entire 2015-2018 training set before evaluation on the 2019 test set.

We opted for a rolling window (walk-forward) CV over expanding window CV because the expanding approach grows the training size set for the later folds, which is computationally expensive given the scale of our data.

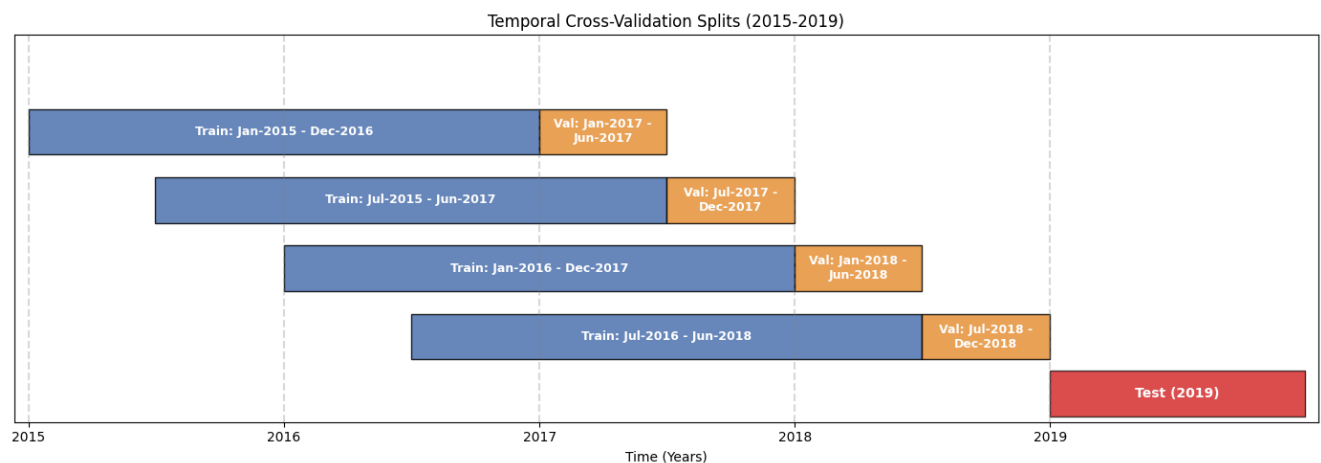


Figure 14. Rolling-window cross-validation strategy

Sampling Strategies

Given that ~80% of flights experience no delays or delays less than 15 minutes, the delay distribution is highly imbalanced. We experimented with several strategies:

1. Oversampling Delayed Flights: Improves model focus on large delays, but increases compute
2. Undersampling non-delayed flights: Reduces bias toward zero but discards potentially useful information
3. Weighted loss functions: Penalizes errors on larger delays without discarding data, but more complicated to implement.

These approaches helped ensure the model learned meaningful patterns across delays of all sizes.

Model Evaluation Metric

Our primary metric for evaluating model results is mean absolute error (MAE):

$$\text{MAE} = \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE weights all errors equally, providing a clear measure of typical prediction error. We also report root mean square error (RMSE) as a secondary metric which penalizes large errors more heavily.

Prior to evaluating MAE, we floored all true and predicted values between 0 and 15 for all models to mitigate the influence of near-zero noise and align evaluation with business relevance.

Loss Functions

The experiments discussed in the next section employed one of the following loss functions:

MSE

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

MSE with Regularization

$$\text{Regularized MSE} = \frac{1}{2N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \left[\alpha \sum_{j=1}^P |\beta_j| + (1 - \alpha) \sum_{j=1}^P \beta_j^2 \right]$$

Pseudo-Huber Loss

$$\text{Pseudo-Huber}(r, \delta) = \delta^2 \left(\sqrt{1 + \left(\frac{r}{\delta} \right)^2} - 1 \right), \text{ where } r \text{ is the residual.}$$

Compute Environment

All experiments were conducted on an m5d.4xlarge, 64GB of memory and 16 cores. It utilized 6-12 workers (auto-scaled) for most of our model experiments. Each worker had 32 GB of memory and 8 cores.

Model Families & Experiments

Baseline: Linear Regression

Table 9. Linear regression results (5 year)

Model	Total Runtime	Feature Set Size	Tuning Iterations	Best Hyper-parameters	Loss Function	Best MAE	Best RMSE
Experiment 1	37min	34	4	Elastic Net: (0.0, 1.0) Regularization: (0.001, 1)	Regularized MSE	Train: 12.962 Test: 14.556	Train: 41.875 Test: 49.383
Experiment 2	2h52m	139	14	Elastic Net: (0.0, 1.0) Regularization: (0.001, 1)	Regularized MSE	Train: 13.001 Test: 14.428	Train: 41.551 Test: 48.926

The initial linear regression experiment (experiment 1) was conducted using only a few select features from the raw data, chosen by intuition. The feature families are as follows:

Table 10. Feature families for baseline model

Feature	Count
Time/Date	7
Airline/Airplane	3
Airport	8
Weather	10

Experiment 2 uses the same feature set as the other models. The model performance improves slightly with engineered and encoded features, suggesting that there are limitations imposed by the model’s linearity assumptions. Despite this, the model provides a reasonable baseline with a 14-minute MAE, while still leaving room for model improvement. These experiments establish baseline performance for the more complex models.

Neural Networks: MLP

We implemented an MLP to capture complex patterns beyond what individual features could explain. The model has three hidden layers with dropout between each layer and weight decay for regularization. High-cardinality categorical variables, such as tail number, were encoded via embeddings, reducing the input dimension to 124. To account for the skewed target distribution, we applied a log transformation to departure delay, as done with the linear model.

The MLP was implemented in PyTorch, which did not scale efficiently across our autoscaling cluster. Attempts to use PyTorch’s distributed framework were unstable, but we were successful in parallelizing the hyperparameter tuning by running independent trials across multiple workers.

Hyperparameter Tuning

We used Ray Tune to parallelize hyperparameter search, following Databricks best practices. Given the large search space, we ran 100 trials per experiment. To tune embeddings without significantly expanding the search space, we introduced a single global scaling parameter that uniformly adjusted embedding sizes across all eight categorical features, with individual dimensions computed using the formula below. Table 11 shows the embedding dimension size for each of our categorical variables.

$$D = \text{Round}(C^{0.25} \times \alpha)$$

where:

- D = Embedding dimension
- C = Vocabulary size
- α = Tunable scaling factor

Table 11. Embedding scaling factor

Feature Name	Vocabulary Size (C)	D ($\alpha=0.75$)	D ($\alpha=1.0$)	D ($\alpha=1.5$)
OP_CARRIER_FL_NUM	7,090	7	9	14
TAIL_NUM	7,839	7	9	14
OP_CARRIER	21	2	2	3
ORIGIN	343	3	4	6
ORIGIN_STATE_ABR	53	2	3	4
DEST	353	3	4	6
DEST_STATE_ABR	53	2	3	4
ORIGIN_PREV	344	3	4	6

Table 12. Parameter search space for MLP experiments

Type	Parameter
Model Architecture	Layer 1 Hidden Dimension: [256-512] Layer 2 Hidden Dimension: [128-256] Layer 3 Hidden Dimension: [64-128] Dropout: [0.2-0.5]
Training	Learning Rate: [1e-4 - 5e-3] Batch Size: [2048, 4096, 8192] Weight Decay: [1e-5 - 1e-3]
Embeddings	Scaling Factor: [0.75, 1.0, 1.5]
Loss Function	Weight Coefficient: [0.25-1] (Experiment 2 only)

All hyperparameter tuning was conducted on 10% of the five-year dataset due to memory and runtime constraints. At this reduced scale, the dataset fits entirely in Ray Tune's in-memory object store, enabling rapid fold loading and supporting up to 84 parallel trials across 12 worker nodes (we left 1 core per worker for ray tune). In contrast, the full dataset exceeded available shared memory capacity, forcing disk-based loading that significantly slowed training. Despite supporting up to 36 concurrent trials, training on the full dataset was prohibitively slow. No trials completed even a single fold after eight hours of execution. This performance degradation stemmed from the combination of slow disk I/O and the model's embedding-heavy architecture, which proved non-parallelizable through multi-threading. Given that architectural hyperparameters such as hidden layer dimensions, learning rate, and regularization transfer well across dataset sizes, we performed hyperparameter search on the 10% sample and applied the optimal configuration to final model training on the complete dataset.

We ran two experiments. In the second, we addressed target imbalance by modifying the loss function to emphasize larger delays. Specifically, loss contributions from flights with delays under 15 minutes were assigned zero weight, while errors for delays exceeding 15 minutes were increasingly upweighted as delay magnitude grew. This approach reflects the decreasing frequency of long delays and encourages the model to focus learning capacity on rarer, high-impact events. The weighting scheme is defined by the equation below and the weighting parameter α was tuned as a hyperparameter. A plot of this equation for various weighting parameters is shown in Figure 15.

$$\mathbf{w}(y) = 1 + \alpha \cdot \max(0, \log(y_{true}) - \log(15+1))$$

This yields a new loss function:

$$L(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n w(y_i) (y_i - \hat{y}_i)^2$$

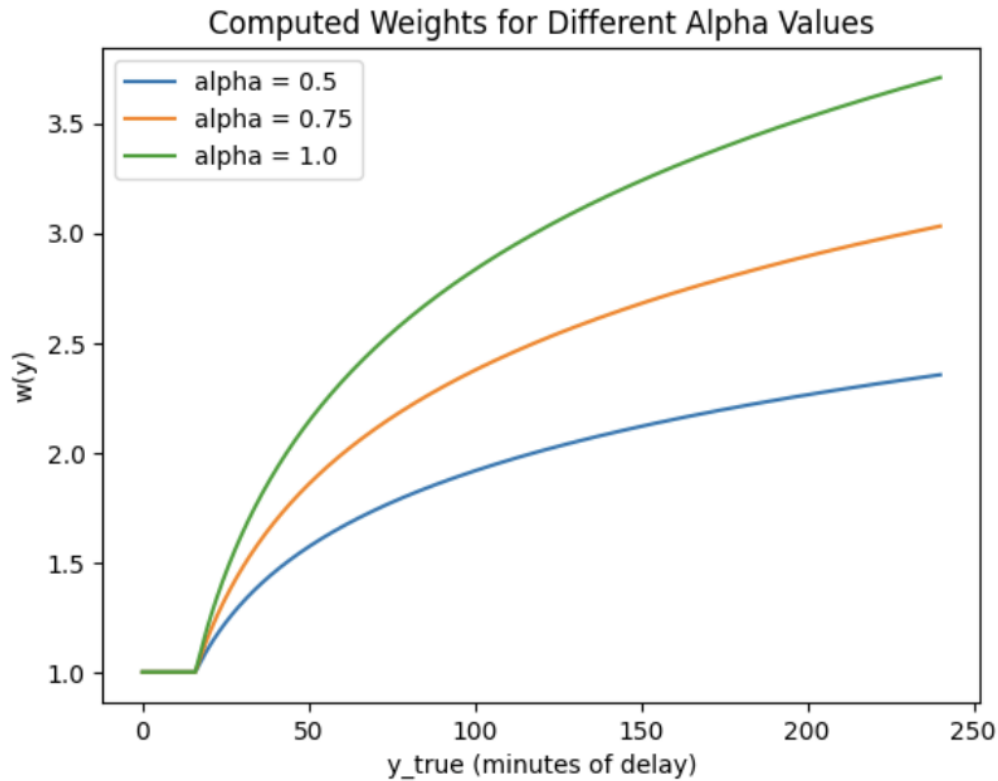


Figure 15. Computed weights for different α values. These weights are then used in the weighted MSE loss function

We ran two trials on 10% of the data and selected the better-performing configuration to train on the full dataset. The weighted and unweighted models performed similarly, with the unweighted model marginally outperforming the weighted version, and was therefore used for the final model. All inferences were done on 100% of the test set.

Table 13. MLP experiment results (5-year data)

Model	Total Runtime	Feature Set Size	Tuning Iterations	Loss Function	Best MAE	Best RMSE
Experiment 1: Unweighted	12 hours (50 concurrent trials)	124	99	MSE	Train: 9.884 Test: 12.943	Train: 44.274 Test: 47.900
Experiment 2: Weighted	8 hours (80 trials at a time)	124	102	Weighted MSE	Train: 10.135* Test: 11.935	Train: 37.540* Test: 46.086

*These values were calculated on a 10% random sample (sampled uniformly from each month)

The final training was done on 100% of the data using 2015 to June 2018 as the train set, June 2018 - December 2018 as the validation set and 2019 as the test set. This was done on the unweighted model due

to it having a lower validation error and the results are in the table above. Due to the large dataset size (30GB preprocessed) exceeding available RAM (64GB), we implemented a mini-batch streaming approach. The preprocessed training data was stored in Parquet format on disk storage. We developed a custom PyTorch IterableDataset that reads data incrementally using PyArrow's row-group iteration. The dataset streams batches directly from Parquet files without loading the entire dataset into memory. PyTorch's DataLoader with 4 worker processes, maintaining a low memory footprint while processing 20 million training samples.

Tree-Based Models: XGBoost

We conducted a series of experiments using XGBoost to evaluate the impact of different hyperparameter configurations, loss functions, model architectures, and sampling strategies on flight delay prediction performance. A key challenge in this task is the severe class imbalance in the target variable: approximately 80% of flights experience no delay or delays shorter than 15 minutes, while large delays are relatively rare but operationally important.

The following table summarized our iterative approach to addressing this imbalance and improving predictive performance, particularly for large-delay events.

Table 14. XGBoost experiment results (5 year)

Model	Total Run-time	Feature Set Size	Tuning Iterations	Hyperparameters Settings	Loss Function	Best MAE	Best RMSE
Single Stage							
Baseline XGB	52 min	139	5	max_depth: (4, 12) num_round: (10, 30) eta: (0.005, 0.02) subsample: (0.6, 1) colsample_bytree: (0.6, 0.9) reg_lambda: (0.0, 0.5) reg_alpha: (0.0, 0.5)	MSE	Train: 14.699 Test: 16.290	Train: 34.124 Test: 41.067
Experiment 1	1.2h	139	10	max_depth: (4, 15) num_round: (10, 20) eta: (0.005, 0.02) subsample: (0.6, 1) colsample_bytree: (0.6, 0.9) reg_lambda: (0.0, 0.5)	MSE	Train: 15.101 Test: 16.692	Train: 34.029 Test: 41.237

Model	Total Run-time	Feature Set Size	Tuning Iterations	Hyperparameters Settings	Loss Function	Best MAE	Best RMSE
				reg_alpha: (0.0, 0.5) gamma: (0.5, 5)			
Experiment 2	-	139	8	max_depth: (4, 12) num_round: (10, 30) eta: (0.005, 0.02) subsample: (0.6, 1) colsample_bytree:(0.6, 0.9) reg_lambda: (0.0, 0.5) reg_alpha: (0.0, 0.5) gamma: (0.5, 5)	Pseudo-Huber	Train: 9.943 Test: 11.305	Train: 36.154 Test: 43.454
Two Stage							
Experiment 3	5.7h	139	30	max_depth:(4, 12) num_round: (10, 30) eta: (0.005, 0.02) subsample: (0.6, 1) colsample_bytree: (0.6, 0.9) reg_lambda: (0.0, 0.5) reg_alpha: (0.0, 0.5)	MSE	Train: 13.271 Test: 14.979	Train: 32.281 Test: 40.384
Experiment 4	1.7h	139	5	max_depth: (4, 12) num_round: (10, 30) eta: (0.005, 0.02) subsample: (0.6, 1) colsample_bytree: (0.6, 0.9) reg_lambda: (0.0, 0.5) reg_alpha: (0.0, 0.5)	MSE	Train: 10.059 Test: 11.464	Train: 35.827 Test: 43.422
Experiment 5	11.8h	139	10	max_depth: (8, 16) num_round: (8, 30) eta: (0.005, 0.02) subsample: (0.6, 1)	Pseudo-Huber	Train: 8.4603 Test: 9.5064	Train: 33.511 Test: 40.762

Model	Total Run-time	Feature Set Size	Tuning Iterations	Hyperparameters Settings	Loss Function	Best MAE	Best RMSE
				colsample_bytree: (0.6, 0.9) reg_lambda: (0.0, 0.5) reg_alpha: (0.0, 0.5)			

Baseline XGBoost

We implemented a baseline XGBoost model with a limited hyperparameter search using Optuna and MAE for evaluation. This simple model slightly improved MAE and RMSE relative to the linear baseline.

Experiment 1: Undersampling and Oversampling

We began with a single-stage XGBoost regression model trained using mean squared error (MSE) as the loss function. Under this baseline configuration, the model achieved a Mean Absolute Error (MAE) of 14.7 minutes on the training set and 16.3 minutes on the test set (see first row of Table 14).

Inspection of residual plots revealed that a substantial portion of the prediction error is driven by flights with large delays. This suggests that the regression model is dominated by the dense mass of short-delay observations and fails to sufficiently learn patterns associated with extreme delay events.

Motivated by this observation, we experimented with a resampling strategy in which non-delayed flights are undersampled by 25%, while delayed flights are oversampled by 25% within each cross-validation fold during hyperparameter tuning and training. The goal is to increase the relative influence of large-delay observations during learning. However, this approach did not lead to meaningful improvements in either training or test MAE, indicating that simple resampling alone is insufficient to address the heavy-tailed nature of the delay distribution.

Experiment 2: Pseudo-Huber Loss

Next, we trained the XGBoost model with uniform sampling and replaced MSE with the pseudo-Huber loss. The pseudo-Huber loss reduces the disproportionate influence of outliers during training, allowing the model to better capture large-delay patterns. This change produced roughly a 30% improvement in both training and test MAE, demonstrating substantially better overall fit (Table 14).

Experiment 3: Parallel Classifier-Regressor Architecture (Two-Stage Model)

To address the class imbalance, we implemented a two-stage framework. The first stage is a binary classifier predicting whether a flight will be delayed more than 15 minutes. The second stage is a regression model predicting the delay magnitude, only trained on delayed flights. We used uniform sampling in training due to computational constraints, though ideally a class balancing strategy would have been employed to give the model more exposure to large delays. This architecture modestly improved MAE over the single stage baseline by reducing noise from the many on-time or minimally delayed flights.

Nonetheless, large delays remain systematically underpredicted, with errors concentrated in the upper tail of the distribution.

A limitation arises because the classifier and regressor train independently. Conflicts occur when the classifier predicts a delay, but the regressor outputs a value below 15 minutes. The training process does not penalize these inconsistencies, leaving some errors unaddressed in the final prediction.

Experiment 4: Joint Evaluation Two-Stage Model

To fix this, we tune hyperparameters using the MAE of the final gated prediction rather than the individual classifier or regressor performance. During inference, we use the regression output only when the classifier predicts a delay; otherwise, we assign zero delay. This joint evaluation aligns the optimization with end-to-end performance. It penalizes classifier-regressor misalignment and encourages parameter choices that consistently predict both the occurrence and magnitude of delays. This further reduced training MAE by an additional ~15% on top of the improvement from changing loss function to pseudo-Huber loss.

Results & Discussion

Table 15. Results of Selection from Each Model Type

Model	Train MAE (minutes)	Test MAE (minutes)	Train RMSE (minutes)	Test RMSE (minutes)
Linear Regression (Baseline)	12.962	14.556	41.875	49.383
MLP	10.594	11.935	37.540	46.086
XGBoost (Single Stage)	9.943	11.305	36.154	43.454
XGBoost (Two Stage)	8.460	9.506	33.511	40.762

The linear baseline performed surprisingly well. Nonetheless, we found that the tree-based XGBoost model improved over the linear model due to its ability to partition the feature space into discrete regions at each split-point. This is particularly powerful for predicting flight delays given the unbalanced, zero-inflated nature of the target feature.

The MLP was designed to capture nonlinear interactions beyond linear and tree-based models using deep hidden layers and embeddings for high-cardinality categorical variables. While this architecture reduced input dimensionality, training did not scale efficiently across our autoscaling cluster, requiring hyperparameter tuning on a 10% data subset. These hyperparameters transferred well to full-data training, but incorporating a weighted loss to emphasize rare, long delays did not improve performance, with the unweighted model performing marginally better. Despite careful tuning and implementing a streaming pipeline to speed up training, the MLP underperformed relative to gradient-boosted trees, underscoring the challenges of applying deep models to large-scale, imbalanced datasets.

The final two-stage model achieved an MAE of 8.46 minutes on the training set and 9.51 minutes on the test set, representing an approximately 35% decrease from our linear baseline. To better understand the remaining errors, we bucketed prediction errors by true delay duration and examined their error distributions (see Figure 16). The plot groups delayed flights into four categories, with the light blue bucket corresponding to delays exceeding five hours. Consistent with intuition, buckets with longer delays exhibit larger average errors, as indicated by the mean of each error distribution. Across all models, the largest prediction errors are concentrated among extreme delays, a pattern that persists in the final model. This behavior is partly attributable to the highly skewed and heavy-tailed distribution of flight delays.

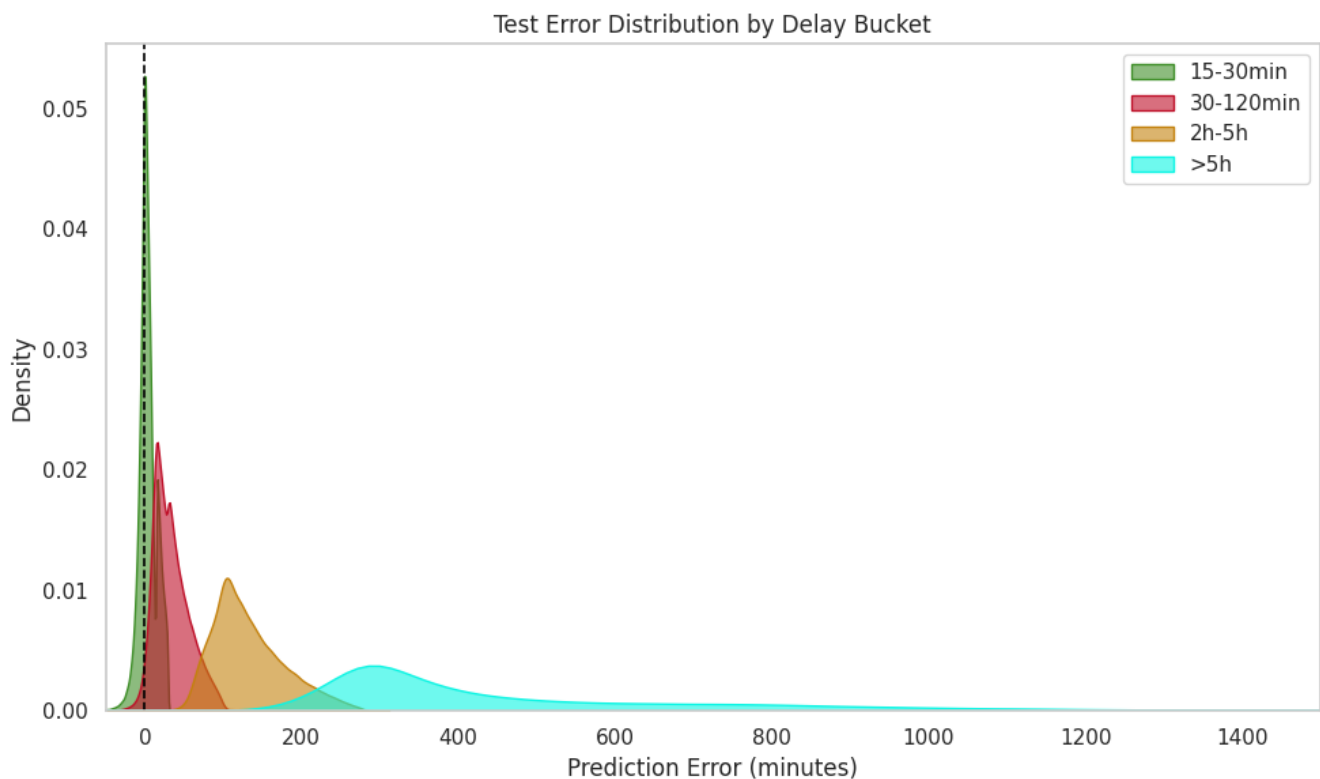


Figure 16. Plot of residuals grouped by (actual) delay

Gap Analysis

Although our dataset spans five years of U.S. domestic flights, extreme delays remain rare, producing a heavy-tailed target distribution. Even with resampling and log transformation, models tend to underpredict very long delays, reflecting limited signal in the tail. In addition, several weather and operational features rely on temporal aggregation or lagging, which constrains the model's ability to respond to sudden, high-impact events such as rapidly developing storms or cascading airport disruptions. Future work could integrate higher-resolution weather data and real-time flight updates (instead of strictly 2-hour forecasts).

As we saw in Figure 16, although the two-stage XGBoost model achieves the lowest MAE, it continues to struggle with large delays. This limitation may affect travelers experiencing unusually long delays, as the model tends to poorly capture these extreme cases. In addition, the classifier-regressor architecture independently optimizes classification and regression objectives, which can produce inconsistent predictions that are not jointly penalized. Alternative approaches—such as unified objective functions or composite evaluation metrics that jointly weight classification performance (e.g., F1) and regression

accuracy (e.g., MAE)—may better capture tail behavior. In addition, the error distribution analysis suggests that clustering-based models could be beneficial, as the error distributions become closer to normal when flights are segmented into more homogeneous delay buckets.

From a systems perspective, training the two-stage model is computationally intensive, particularly under rolling-window cross-validation. Although this approach scales to our dataset, extending it to larger time horizons or a bigger dataset would require more efficient distributed training.

Finally, despite achieving a best MAE of 9.5 minutes on our test set, there remains substantial opportunity for improvement. Future extensions could emphasize temporal adaptation through recency-weighted training or ensemble methods that combine models trained on different cross-validation folds, improving robustness to evolving traffic and weather patterns.

Conclusion & Future Work

We built a machine learning model to predict U.S. flight delays two hours before scheduled departure, with the hypothesis that engineered features improve the predictive performance of our regression model. We can use this model to alert travelers to potential flight delays greater than 15 minutes. Key feature families driving performance were flight history, graph-based airport connectivity, and weather-derived features. In particular, the delay of the previous flight showed particularly high correlation with the target feature. The final two-stage XGBoost model achieved a test MAE of 9.51 minutes, substantially improving over the linear baseline (14.55 minutes), and providing a realistic estimate for actual flight departure time.

Future work includes integrating higher-resolution and real-time weather data, weighting recent observations more heavily, and exploring ensembles trained on CV folds to improve extreme-delay predictions. Considering distributed or incremental training would improve scalability for larger datasets or higher-frequency features.

Phase Leader Plan

Table 1. Phase Leader Plan

Week	Phase	Phase Leader	Dates
1	Phase 1: Project Plan, describe datasets, joins, tasks, and metrics	Emily	Oct 27 - Nov 2
2	Phase 2: EDA, baseline pipeline, Scalability, Efficiency, Distributed/parallel Training, and Scoring Pipeline	Maggie	Nov 3 - Nov 9
3	Phase 2 continued	Lyn	Nov 17 - 23
4	Phase 3: Select the optimal algorithm, fine-tune and submit a final report	Michael	Dec 1 - Dec 7
5	Phase 3 continued	Aidan	Dec 8 - Dec 15

Credit Assignment Plan

Table 16. Credit assignment

Team member	Task Description	Estimated Hours	Actual Hours	Start Date	End Date	Status
Everyone	Group meetings covering planning, coding, and writing and editing the phase report / presentation. Also other small misc tasks like debugging.	25	35	11-26	12-14	C
Aidan	Built scaffolding for a 2-stage model (MLP → XGBoost with MLP as classifier)	7	7	12-2	12-4	C
Aidan	Experiment with sampling strategies on 1y data	5	5	12-3	12-4	C
Aidan	Cyclical encoding and graph feature engineering	2	2	12-5	12-5	C

Team member	Task Description	Estimated Hours	Actual Hours	Start Date	End Date	Status
Aidan	Wrote abstract, introduction, pipeline overview, conclusion. Helped with overall formatting.	5	5	12-13	12-14	C
Emily	Explore embeddings viability	1	2	12-2	12-3	C
Emily	Transform some categorical features using binary and sinusoidal to reduce dimensionality	1	1	12-3	12-4	C
Emily	Modify and run baseline for 5year data	1	1.5	12-7	12-9	C
Emily	Write phase report (solo/offline, multiple sections, diagrams, and worked with team to format and edit)	8	14	12-10	12-14	C
Lyn	Implemented post-processing for custom-joined datasets and data preparation for our 3rd phase models	2	2	12-2	12-3	C
Lyn	Engineered network-based features, including three degree-centrality metrics and time-based outbound traffic feature	2	3	12-2	12-4	C
Lyn	Built and evaluated cross-validation pipelines for XGBoost and MLP models with 5 year data	1	1	12-3	12-4	C
Lyn	Designed, executed and analyzed 12 hyperparameter-tuning experiments across single-stage and two-stage XGBoost models	15	40	12-4	12-14	C
Lyn	Wrote modeling-XGBoost section, results and discussion, assisted with gap analysis	2	5	12-13	12-14	C
Michael	Engineered the weather type features: Rain, Snow, Hail, etc. Required parsing REM and additional columns	4	10	11-30	12-09	C

Team member	Task Description	Estimated Hours	Actual Hours	Start Date	End Date	Status
Michael	Engineered the holiday and holiday-window features	4	3	12-4	12-09	C
Michael	EDA on numerical weather features with spearman correlation matrix	3	5	12-4	12-14	C
Michael	Outlined MLP model using PyTorch (but used Maggie's)	2	2	12-4	12-09	C
Michael	Wrote leakage analysis, model evaluation metrics and loss function sections, plus assisted with drafting and editing many sections	8	20	12-10	12-14	C
Maggie	Weather Feature Engineering	5	5	11-27-25	12-4-25	C
Maggie	Flight time-based feature engineering	5	8	11-27-25	12-4-25	C
Maggie	MLP model framework	5	3	12-4-25	12-7-25	C
Maggie	MLP parallelization	20	50	12-4-25	12-10-25	C
Maggie	MLP embeddings	5	4	12-8-25	12-9-25	C
Maggie	MLP Weighted loss function	2	2	12-8-25	12-9-25	C
Maggie	MLP Tuning Training Time	20	20	12-8-25	12-14-25	C

Team Members:

- Aidan Connerly: aidanconnerly@berkeley.edu
- Maggie Royal: mroyal7@berkeley.edu
- Fuling Wang: fulingw@berkeley.edu
- Michael White: msteckwhite@berkeley.edu
- Emily Zhao: emily_zhao@berkeley.edu

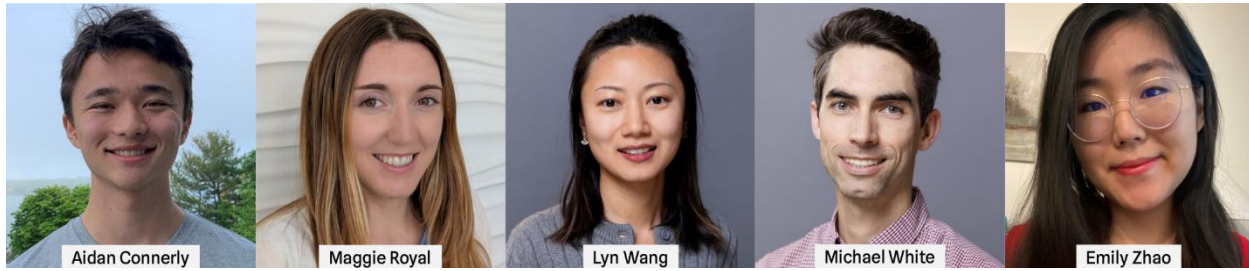


Figure 16. Group 3-1 Team Members